Evolutionary Computation

Luiz Gustavo DE LIMA PREVIERO
Zizhen WANG

7 May 2026

Institut Supérieur de l'Aéronautique et de l'Espace
SUPAERO

Contents

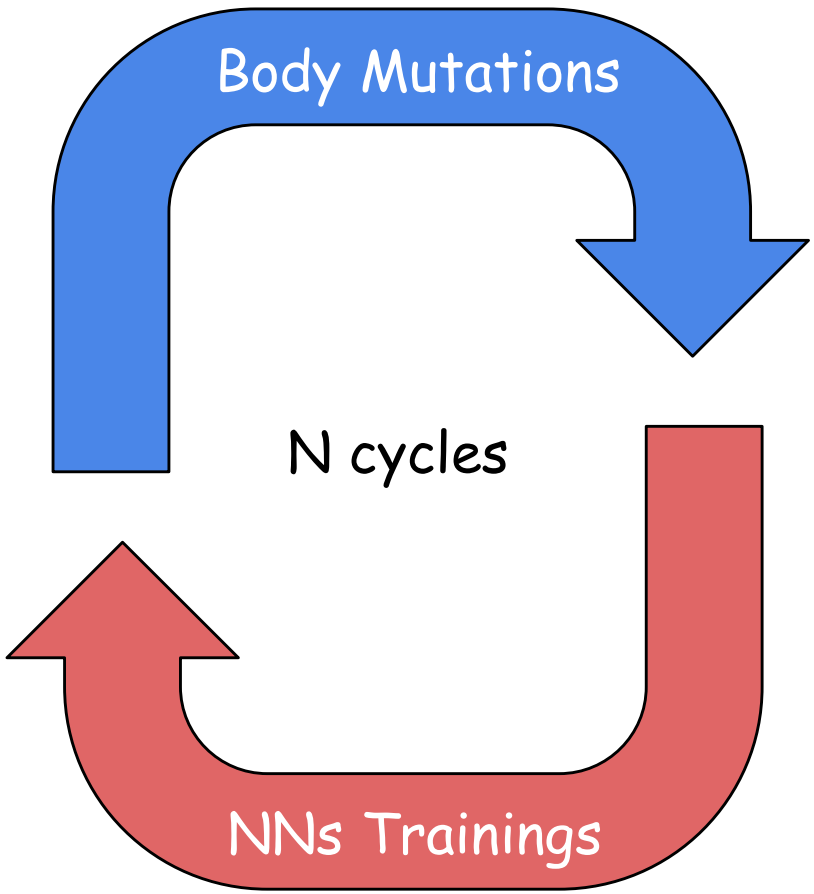
1 Code Architecture & Snippets

2 Setting Up a Simulation

3 Task Results (Walker - Thrower - Climber)

Code Architecture & Snippets

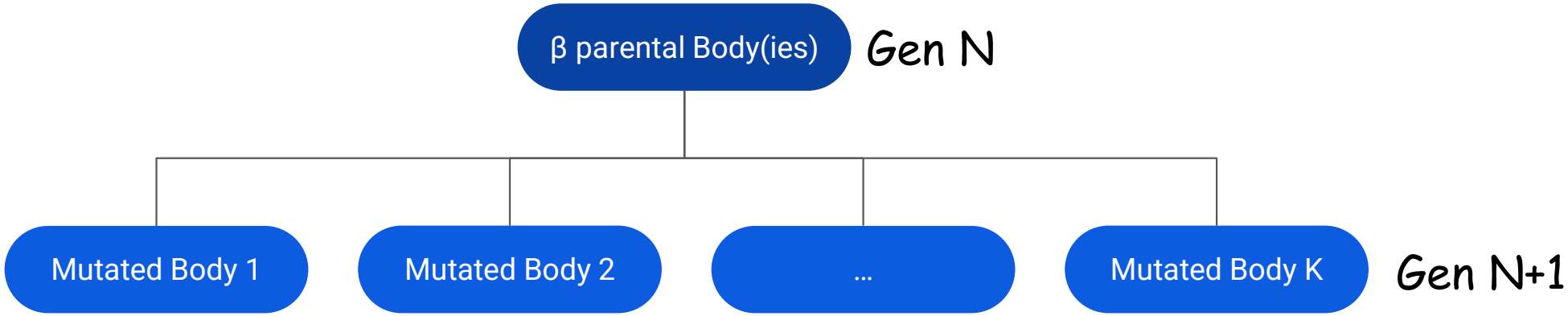
- General Structure is based on :
  - Create mutations of the parental body;
  - Train a NN for each mutated body;
  - Save graphs/.json/GIFs;
  - Pass the best bodies forward;
  - Repeat this an N amount of cycles;
- Files restructured from original repository :
  - 1 class for Environment functions
  - 1 class for Neural Network functions
  - 1 class for Utility functions
  - 1 class for controlling NN evolution
  - 1 class for controlling Body Evolution
  - 1 main function with setup configuration



CONFIDENTIAL

Code Architecture & Snippets

- Body evolution is achieved through N cycles of an ((1/β < α) + K) ES :
  - N is read from the size of the evolution map (explained later);
  - β is the amount of bodies better than the parental one, capped at fixed α;
  - K is chosen as a parameter (4 by default, 10 for "super mutators");
  - The amount of mutations per body is also a customizable parameter;
  - All bodies are contained in a 5x5 grid (not necessarily full);



Code Architecture & Snippets

- About the *mutate* function and its dependencies :
  - np.randint selects a cell within the 5x5 grid of the body;
  - np.randint then proposes a tissue type that's different from the current one in the cell:
    - 0 - empty, 1 - hard block, 2 - soft block, 3 - H. muscle, 4 - V. muscle;
  - The *unicity_check* function is called to assure that the mutation will keep the body unique/continuous (need of the EvoGym environment);
    - If the check succeeds, then the modification is committed, if it fails, the code is re-run to place the same proposed tissue elsewhere;
  - Last check to assure the presence of at least one muscle cell in the body (need of the EvoGym environment).

ISAE-SUPAERO

Code Architecture & Snippets

```python
def mutate_body(body, mutation_amount = 5, forceType = False, c = 0, show = False):

        mutated_body = np.copy(body)

        for _ in range(mutation_amount):

            i = np.random.randint(0,5)
            j = np.random.randint(0,5)

            if forceType:
                k = c
            else:
                current = mutated_body[i][j]
                allowed_values = np.setdiff1d(range(0, 5), current)
                k = np.random.choice(allowed_values)

            willBreak = unicity_check(mutated_body,i,j,k)

            if willBreak:
                mutated_body = mutate_body(mutated_body, mutation_amount = 1, forceType = True, c = k)
            else:
                mutated_body[i][j] = k

        if((3 not in mutated_body) and (4 not in mutated_body)):
            mutated_body[2][2] = 3

        if show:
            print(mutated_body)
        return mutated_body
```

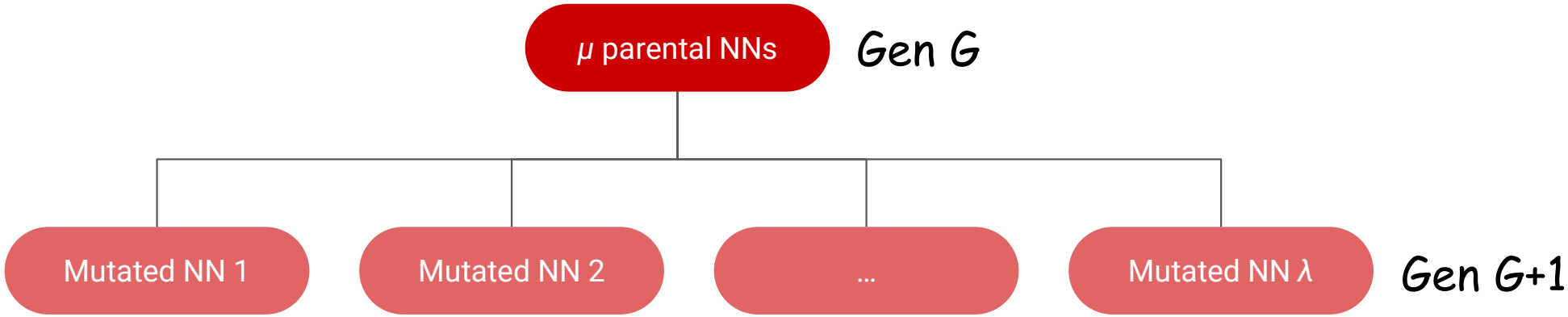
Code Architecture & Snippets

- About the recombination of β better parental bodies for next generation :
  - Allows to better used the bodies evolved during the previous cycle;
  - β is updated dynamically, and capped at α = 2 or 1 for most phases (avoiding the creation of Frankensteins…);
  - The mutations of the bodies (in relation to their common parent) are superposed, with priority to the better one;
  - *A body_sewing* algorithm is then called to make sure the process don't leave any gaps;

```
mutation_stack = np.zeros((5,5), dtype=int)

cur_alpha = alpha[map[cycle]]

for i in range(np.min([cur_alpha,beta])):

    mutation = test_bodies[idx[cur_alpha-1-i]]-elite_body
    mask = mutation != 0
    mutation_stack[mask] = mutation[mask]

elite_body += mutation_stack

elite_body = body_sewing(elite_body)
```

Code Architecture & Snippets

- Similarly, each NN evolution is achieved through G cycles of an (μ + λ) ES :
  - μ is the size of the parental population (5 by default);
  - A logarithm weight array is applied to the μ parent's genes;
  - λ is selected from the profiles by the evolution map (similar to G);
  - The simulations of the children are parallelized among the different CPU cores (greatly speeding up the process)



Setting Up a Simulation

- Done through the configuration of the *Main* file :
  - Choice of the initial body (possibility of starting from zero with a body template or resuming the evolution of a previous body);
  - Choice of EvoGym environment of simulation;
  - Customization of different parameters for the cycle profiles, being :
    - Profile 0 for initial wide range search of promising solutions (faster);
    - Profile 1 for intermediate improvement of current solutions (medium);
    - Profile 2 for specialization of the best solutions found (slower);
  - Definition of an evolution map (simple array with 0,1 and 2's) that will pilot the algorithms during the body cycles.

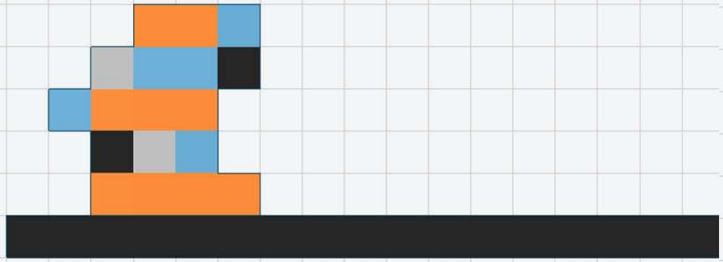
Setting Up a Simulation

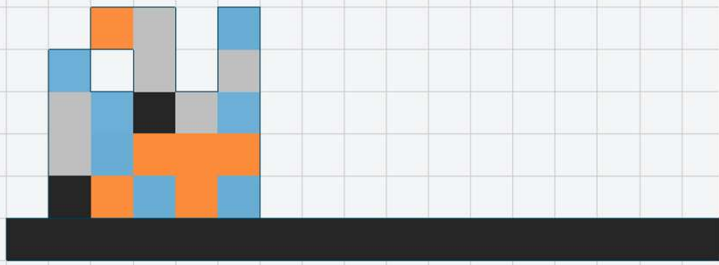
```python
def main():
    print("Hello from project!")

    initial_body = BodyEvolution.initialize_default_full()

    print(initial_body)

    setup = {
        # Environment and robot definition;
        "env_name": "Walker-v0",
        "body": initial_body,
        # Body evolution parameters;
        "amount": 4,
        "map" : [0,0,0,0,1,1,1,1,2,2,2,2],
        "mutations" : [5,4,3],
        "alpha": [2,1,1],
        # Brain evolution parameters;
        "generations": [40,80,400], # to change: increase!
        "lambda": [50,40,25], # Population size
        "mu": 5, # Parents pop size
        "sigma": 0.1, # mutation std
        "lr": 1.25, # Learning rate
        "max_steps": 500, # to change to 500
        "folder": "Walker2/",
    }

    Utilities.initial_warning(setup)

    BodyEvolution.one_plus_lambda(setup, save_graphs = True, save_anims = True, save_solutions = True)
```

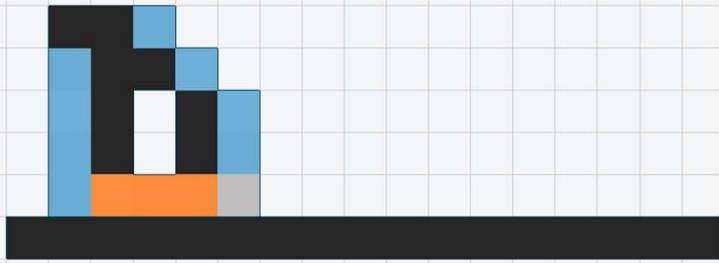
Setting Up a Simulation

```python
def main():
    print("Hello from project!")

    initial_body = np.array([
    [0, 0, 1, 3, 0],
    [3, 4, 2, 2, 1],
    [4, 3, 3, 2, 0],
    [0, 2, 2, 4, 0],
    [0, 4, 4, 3, 3]
    ])

    print(initial_body)

    setup = {
        # Environment and robot definition;
        "env_name": "Walker-v0",
        "body": initial_body,
        # Body evolution parameters;
        "amount": 4,
        "map" : [1,1,1,2,2,2],
        "mutations" : [5,4,3],
        "alpha": [2,1,1],
        # Brain evolution parameters;
        "generations": [40,80,400], # to change: increase!
        "lambda": [50,40,25], # Population size
        "mu": 5, # Parents pop size
        "sigma": 0.1, # mutation std
        "lr": 1.25, # Learning rate
        "max_steps": 500, # to change to 500
        "folder": "Walker1/",
    }

    Utilities.initial_warning(setup)

    BodyEvolution.one_plus_lambda(setup, save_graphs = True, save_anims = True, save_solutions = True, old_fitness = 5.8148518, old_total_eval = 49600)
```

Task Results (Walker)

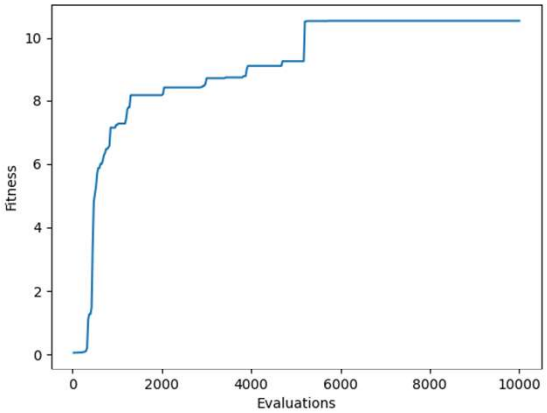




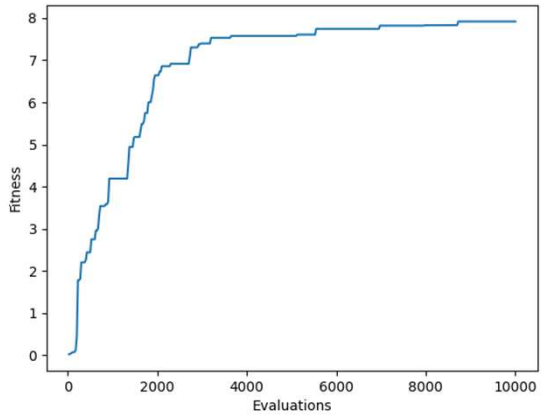


Task Results (Walker)

Fitness : 10,54

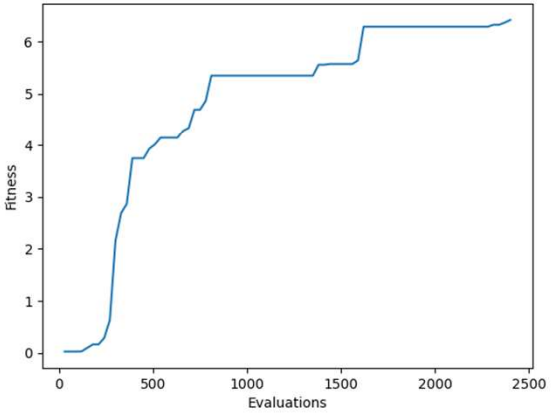


Fitness : 7,92

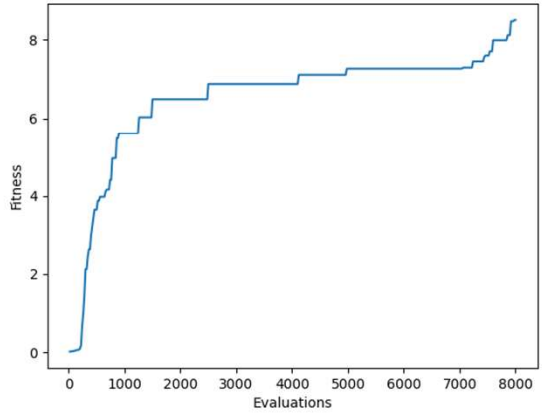


Two-Best Average Fitness : 9,52

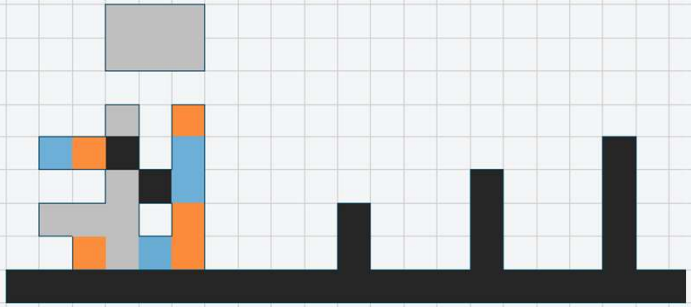
Fitness : 6,41

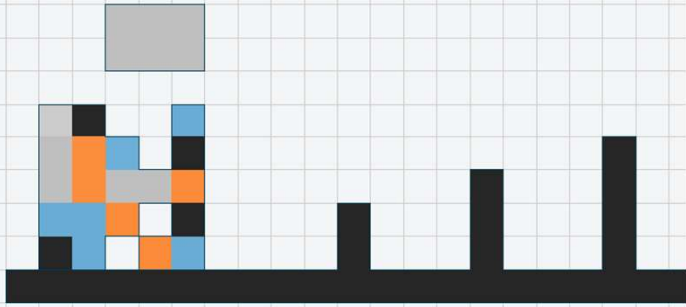


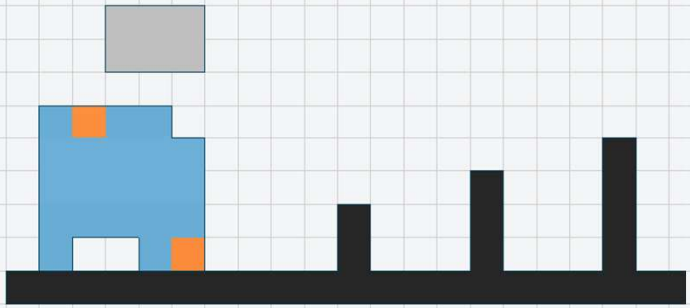
Fitness : 8,50

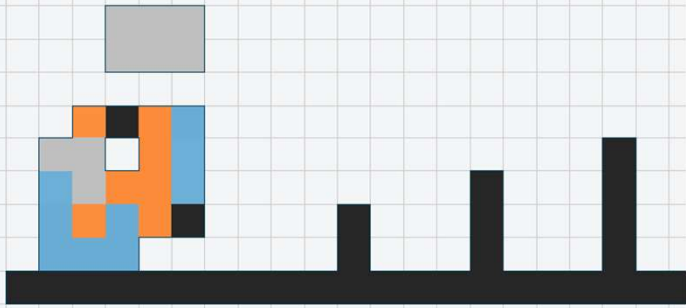


Task Results (Throwers)



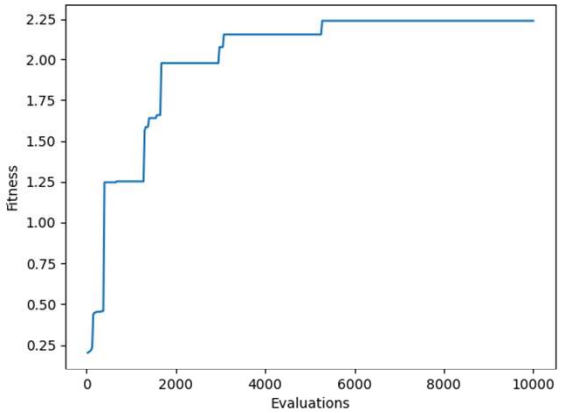




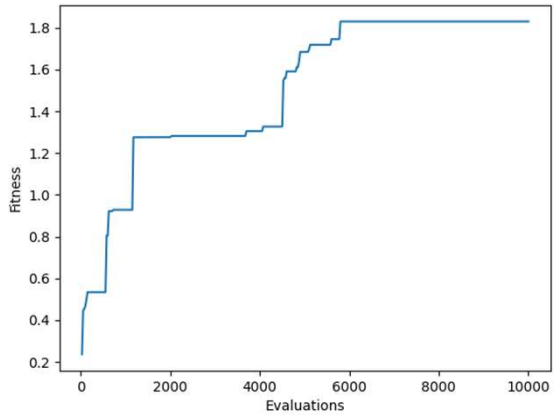


Task Results (Throwers)

Fitness : 2,24

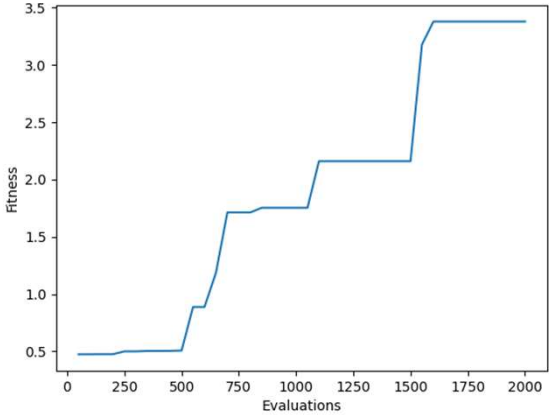


Fitness : 1,83

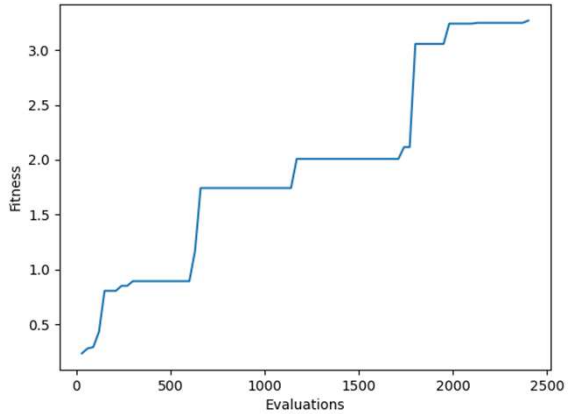


Two-Best Average Fitness : 3,33

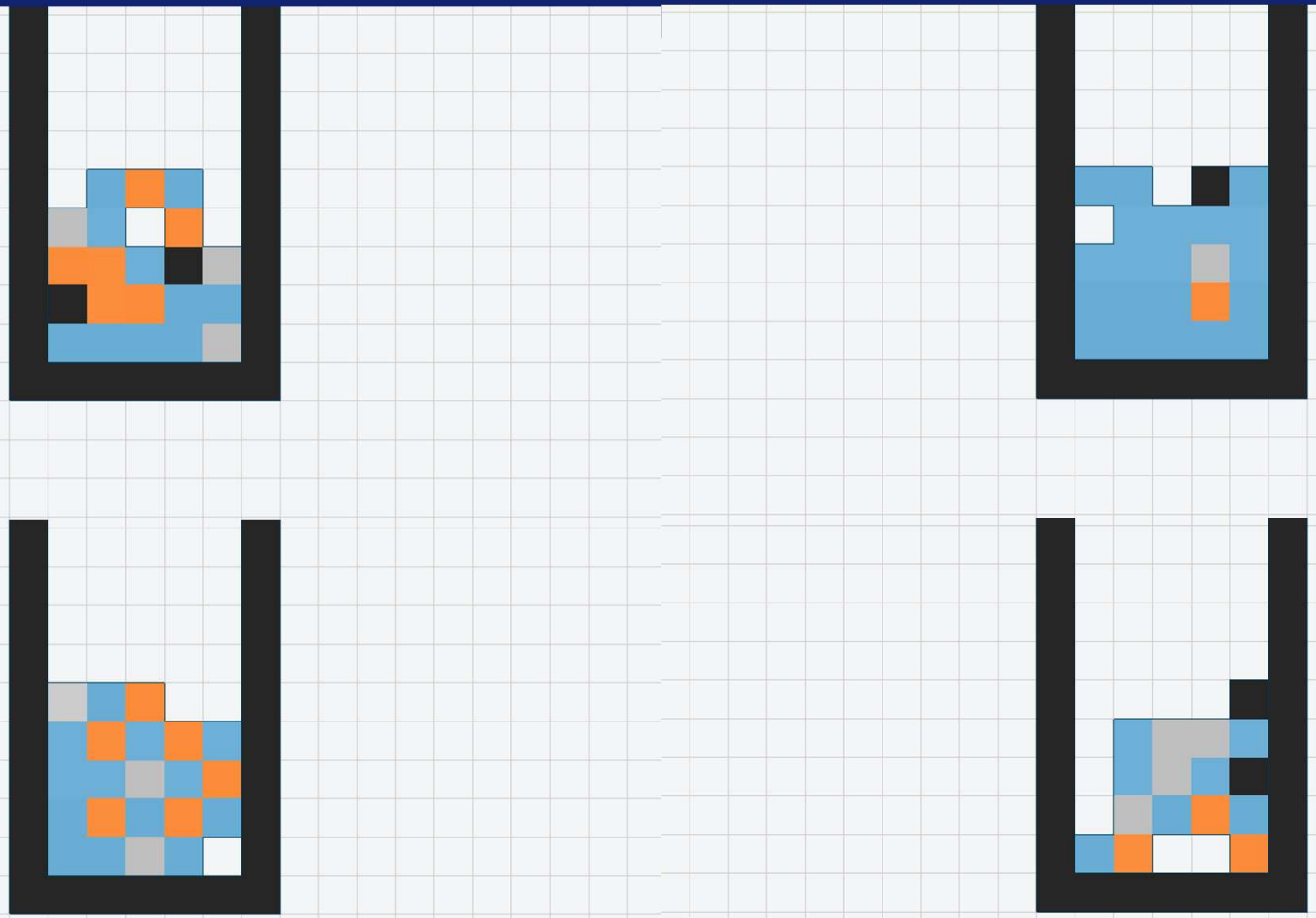
Fitness : 3,38



Fitness : 3,27

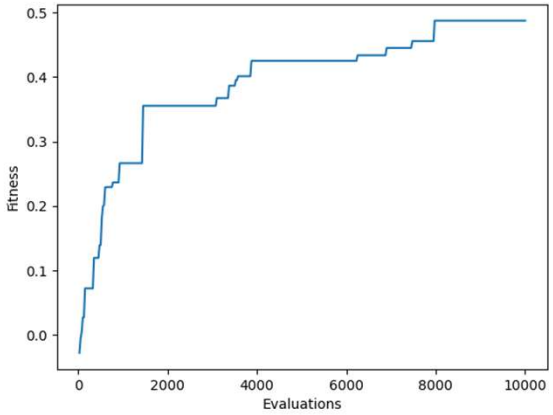


Task Results (Climbers)

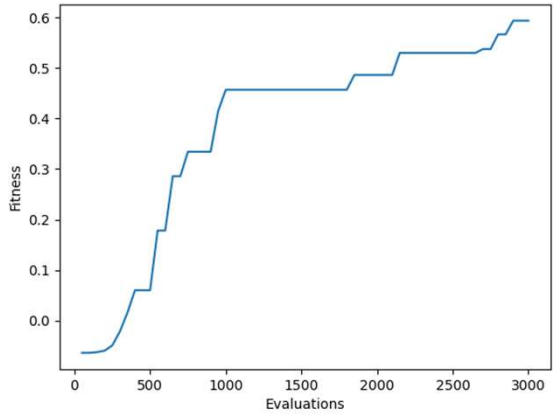


Task Results (Climbers)

Fitness : 0,49

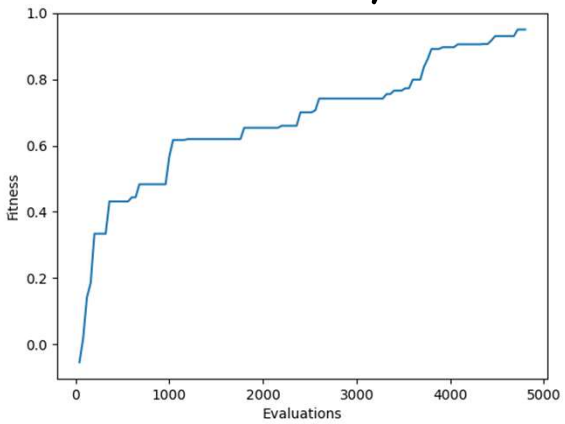


Fitness : 0,59



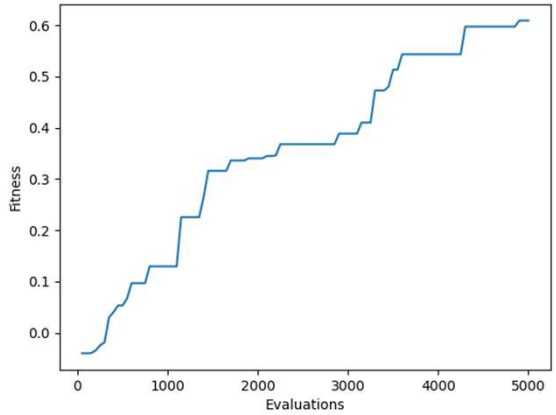
Two-Best Average Fitness : 0,78

Fitness : 0,95



ISAE-SUPAERO

Fitness : 0,61



Things to Improve

Thanks for your attention !

Luiz Gustavo DE LIMA PREVIERO
Zizhen WANG

7 May 2026

Institut Supérieur de l'Aéronautique et de l'Espace
SUPAERO

Questions & Answers